# Auditoría técnica UEP v0.5.6

Revisión de ingeniería del paquete `UEP-Universal-Economic-Protocol-0_5_6.zip` (679 archivos, TypeScript sin dependencias + crates Rust de laboratorio). Fecha: 9 de octubre de 2026.

## 1. Alcance y método (leer primero)

**Qué hice**

- Extraje y leí el código criptográfico y de identidad ( `src/core/hash.ts` , `note.ts` , `field.ts` , `spend-key.ts` , `security-policy.ts` , `network-tag.ts` , `poseidon-wasm.ts` ), la identidad/wallet ( `src/identity/*` ), la ruta de mint y faucet del ledger, la API HTTP ( `src/service/uep-http-api.ts` ), el CI y la documentación de estado.
- Ejecuté `npm run test:protocol` (210 tests), `lint:determinism` , `check:version` , `check:snapshot-compat` y `check:donation` .
- Reproduje con una prueba de concepto el hallazgo C1 y escribí y probé su corrección.

**Qué NO hice (importante)**

- No pude compilar ni ejecutar Rust (no hay `cargo` en mi entorno): los circuitos, `uep-core/*` y `test:rust` quedan sin ejecutar.
- No ejecuté los labs ( `test:lab` ), ni la simulación de 20k usuarios.
- No revisé línea a línea `ledger.ts` (2.458 líneas), `marketplace.ts` (136 KB), settlement, oracle ni category. Quedan fuera de este informe.
- Los hallazgos marcados **por inspección** no están reproducidos; vuestros ingenieros deben confirmarlos antes de actuar.

Esto es una revisión técnica asistida, no sustituye una auditoría externa formal ni la revisión criptográfica del circuito.

## 2. Resumen ejecutivo

El proyecto está bien estructurado: cero dependencias, snapshots versionados con migraciones y fixtures, vectores de prueba públicos (RFC 8032, 9162, 9180, 5869), Ed25519 estricto y documentación honesta sobre límites. Los puntos críticos de esta versión:

1. **`supplyCap` vulnerable** (confirmado): se pueden emitir más unidades que el tope usando reservas de mint simultáneas. Parche incluido.

2. **`npm test` y el CI fallan en v0.5.6** (confirmado): el lint de determinismo detecta 3 violaciones introducidas en 0.5.4 y 0.5.6, y 2 tests fallan por eso. El README sigue afirmando estado verde.
3. **DoS por `BigInt` en la API HTTP** (confirmado): un cuerpo de hasta 16 MiB bloquea el event loop varios segundos.
4. **Diseño no determinista en el ledger** (por inspección): el ledger genera aleatoriedad internamente, incompatible con ejecución replicada.
5. **Módulo WASM de Poseidon no compilable tal como se genera** (por inspección): usa `u128` para un campo de 254 bits.
6. **Vault del wallet débil frente a fuerza bruta offline** (por inspección): PIN + PBKDF2 de 100.000 iteraciones en `localStorage`.

## 3. Hallazgos confirmados (reproducidos)

### C1 · Alto · Tope de emisión ( `supplyCap` ) eludible

`prepareIssuerMint()` comprueba el tope contra la oferta ya emitida, sin contar las reservas pendientes. `mintWithIssuerSignatures()` aplica la reserva sin volver a comprobar tope, saldo máximo ni si el activo sigue siendo emitible. Resultado: con tope 1000, dos reservas de 900 se preparan y se aplican, y el saldo queda en **1800**.

**Reproducción:** prueba de concepto sobre `ledger-issuer-mint.test.ts` (asset `uep-sim/senergy`, 2-de-3 emisores, tope 1000). Falló con `supply cap exceeded`, saldo 1800.

**Corrección (probada):**

- `assertSupplyCap` suma las reservas vivas del mismo activo.
- `mintWithIssuerSignatures` repite, antes de consumir la reserva: `assertMintable`, `assertSupplyCap` sin reservas y la comprobación de saldo `< 2^64`.
- Test de regresión nuevo: `src/testnet/ledger-mint-reservation-cap.test.ts` (2 tests).
- Resultado tras el parche: 2/2 del test nuevo, 3/3 de `ledger-issuer-mint`, 8/8 de `ledger-v054`. No ejecuté de nuevo la suite completa tras el parche.

Patch adjunto: `fix-supplycap.patch`.

### C2 · Alto (proceso) · La suite y el CI están en rojo en v0.5.6

`npm run lint:determinism` termina con código 1 y 3 violaciones:

- `src/testnet/ledger.ts:30` y `:2398`: `randomBytes` (introducido con `ephemeralFr()` en 0.5.4).

- `src/marketplace/scale-sizes.ts:23`: lectura de `process.env.UEP_SCALE` (introducido en 0.5.6).

Dos tests de `test:protocol` fallan por esto (`determinism lint: transition paths…` y `an injected Date.now…`). `test:core` y el paso «Deterministic transitions lint» del CI no pueden pasar. Resultado de mi ejecución: 210 tests, 208 correctos, 2 fallidos.

**Acciones:** (a) sacar la lectura de entorno de `scale-sizes.ts` al script/test que la usa y pasarla como parámetro; (b) resolver la aleatoriedad del ledger (ver I1) en lugar de añadirla a la lista de permitidos; (c) hacer que el pipeline de release bloquee si el CI no está verde.

### C3 · Medio · Bloqueo del event loop con `BigInt` en la API HTTP

Rutas como `POST /v1/marketplace/listings` hacen `BigInt(String(body.unitPrice))` sobre un cuerpo de hasta **16 MiB** (`readBody` por defecto). El parseo de `BigInt` decimal es cuadrático. Medido en Node 22.22: 1,6 M dígitos ≈ 0,26 s; 6,4 M dígitos ≈ 1,5 s; extrapolado a ~16 M dígitos ≈ 10 s por petición, con el servidor sin responder a nadie.

**Acciones:** límite de cuerpo por ruta (la de ledger ya usa 1 MiB; el resto puede ser 16–64 KiB), validar con regex `^[0-9]{1,30}$` antes de `BigInt`, y rechazar números no canónicos.

### C4 · Bajo · Fuga de mensajes internos

El `catch` global de la API devuelve `e.message` en las respuestas 500. Devolved un código genérico y un identificador de correlación; registrad el detalle solo en el log.

## 4. Hallazgos por inspección (por confirmar)

### I1 · Alto · Aleatoriedad dentro del ledger

`ephemeralFr()` usa `randomBytes(32)` para el blinding de faucet y mints. Dos nodos que ejecuten las mismas transacciones obtendrían compromisos distintos y raíces de estado distintas. Es incompatible con ejecución replicada (consenso).

**Propuesta:** que el destinatario o el emisor aporte el compromiso de la nota en la petición (el blinding nunca entra en el ledger); el ledger solo verifica y registra. Eso además elimina `blindingHex` de la memoria del ledger y cierra C2 de forma estructural.

### I2 · Alto · Módulo WASM de Poseidon

`uep-core/uep-poseidon-wasm/build.sh` genera Rust con `const P: u128 = 2188…5617u128` (≈ 2^254) y constantes de ronda en `u128`. Un `u128` llega a ~3,4·10^38, así que esos literales no caben y el código no compilaría; aunque compilara, `x * x` desbordaría y

la función exportada recibe y devuelve `u64`. Además `setPoseidonWasm()` sustituye el backend hash global sin comprobar paridad.

**Propuestas:**

- Aritmética de campo con 4 limbs de 64 bits (Montgomery), no `u128`.
- Al instalar el módulo, comparar `N` vectores (`uep-core/vectors/*`) y entradas aleatorias contra JS; si hay diferencia, no instalar.
- Commit del `.wasm` con su SHA-256 y toolchain fijada, como ya planea `docs/PERFORMANCE.md`.
- Que cambiar de backend tras arrancar el ledger sea un error (`HASH_BACKEND_LOCKED`).

## I3 · Alto · Vault del wallet

`src/identity/vault.ts` guarda mnemónico y semilla cifrados con AES-GCM, clave derivada con PBKDF2-SHA-256 de **100.000** iteraciones a partir de un **PIN**, en `localStorage`. Un PIN de 4–6 dígitos se agota offline en segundos a minutos. Otros puntos: la cabecera (iteraciones, salt) no va como AAD; no hay límite de intentos.

**Propuestas:**

- Pasar a Argon2id o scrypt (WASM, con parámetros fijados) y, como mínimo, PBKDF2 ≥ 600.000 con una frase de paso, no un PIN.
- Guardar la clave de cifrado en el almacén seguro del sistema (Android Keystore / iOS Secure Enclave / WebAuthn PRF) y usar el PIN solo para desbloquear.
- Incluir `version|kdf|iterations|salt|accountId` como AAD.
- No usar `localStorage` para secretos; usar IndexedDB cifrado o almacenamiento nativo.

## I4 · Medio · Derivación de claves de identidad

`deriveIdentity` obtiene `secret` y `salt` con HMAC fijos («UEP v0.1 account secret») y de ahí una única clave Ed25519. No hay índice de cuenta, ni rotación: el id de cuenta compromete esa clave, así que una clave comprometida obliga a migrar fondos.

**Propuestas:** derivación SLIP-0010 para Ed25519 con ruta `m/…'/cuenta'/índice'`; tipo de transacción «rotar clave» con retardo y firma de la clave antigua; recuperación social o multisig umbral (ya tenéis M-de-N para emisores, reutilizable para cuentas).

## I5 · Bajo · BIP-39 sin normalización NFKD

`mnemonicToSeed` no normaliza mnemónico ni passphrase a NFKD como exige BIP-39; con passphrases no ASCII la semilla difiere de otros wallets.

### I6 · Medio · Reservas de mint sin límite

`mintReservations` no tiene tope y el orden es FIFO estricto: quien reserve primero bloquea ( `MINT_RESERVATION_HELD` ) a los demás hasta que expire (12 alturas). Si `prepareIssuerMint` es accesible a no emisores, se puede retener la cola. **Propuesta:** tope de reservas por activo, reserva solo con firma de un emisor y expiración corta.

### I7 · Medio · `SecurityPolicy`

- La ventana «rolling» es fija: se pueden concentrar ~2× el límite alrededor del cambio de ventana.
- `check(probe, false)` llama a `bucket()` , que puede reiniciar ventanas aunque no se confirme (documentado como no mutante).
- Los límites son por cuenta; con cuentas ilimitadas gratis son una mitigación, no un control de seguridad. Mejor ventana deslizante (cubos de N alturas) y límites ligados a coste (fee mínima, depósito).

### I8 · Medio · Superficie de la API HTTP

- La comprobación de `Host` / `Origin` (anti DNS rebinding) solo cubre `/v1/objects` ; el resto de rutas no la aplican.
- `/v1/ledger/transactions` , `/v1/relay/envelopes` , `/v1/compute/jobs` y `/v1/economic/accounts/:id` no exigen autenticación; no hay rate limiting por IP o por actor.
- Sin cabeceras de seguridad ( `X-Content-Type-Options: nosniff` , `Cache-Control: no-store` ) ni `server.requestTimeout` / `headersTimeout` / `maxRequestsPerSocket` explícitos.
- Un único bearer estático para objetos, sin rotación ni alcances.

**Propuesta:** middleware único que aplique Host/Origin, límites de cuerpo por ruta, rate limit (token bucket), timeouts y cabeceras; tokens con alcance y rotación; esquema OpenAPI y validación de entrada con tipos estrictos.

### I9 · Medio · CI y cadena de suministro

- Acciones por etiqueta ( `@v7` , `@v4` ), no por SHA de commit.
- `tsc --noEmit` es **no bloqueante**: todo el código se ejecuta con `--experimental-strip-types` , así que los tipos nunca se comprueban en bloqueante.
- No hay CodeQL, escaneo de secretos, `cargo audit` / `cargo deny` , ni `npm audit signatures` ; sin cobertura, fuzzing ni SBOM/provenance.
- El job `lab-known-issues` está vacío.

**Propuesta:** fijar acciones por SHA, `tsc` bloqueante, CodeQL + gitleaks + `cargo deny`, `cargo build --locked`, y provenance (SLSA) en las releases.

### I10 · Bajo · Documentación desincronizada

El README dice «Current status (v0.5.3)» con «snapshot format 9», pero el paquete es 0.5.6 con formato 10. Las revisiones externas se citan pero sus informes no están publicados. Generad la tabla de estado a partir de `package.json` y `FORMAT.json` para que no diverja.

### I11 · Medio · Dos composiciones de fold conviven

`hFold` trata un singleton como identidad y no liga la longitud. 0.5.4 añade `hFoldV2`, y 0.5.5 congela el circuito en V2, pero el `canonicalTxCommitment` del wallet y los compromisos de nota siguen en `hFold`. El riesgo es de confusión wallet↔circuito, no de rotura inmediata. **Propuesta:** tabla normativa «qué compromiso usa qué fold», vectores cruzados TS↔Rust y fecha de retirada de `hFold`. `CIRCUIT_CONSTRAINTS` sigue siendo la medición v4.

### I12 · Informativo · ZK

Vuestras notas ya lo dicen: claves de desarrollo, sin ceremonia, firma de emisor aún exigida y sin privacidad de remitente/importe. Observación: Groth16 requiere una ceremonia por cada cambio de circuito (v4→v5 en una sola release). Mientras el circuito siga cambiando, conviene un esquema con setup universal (PLONK/Halo2) o posponer toda ceremonia hasta el congelado final.

## 5. Rendimiento

Medición propia de vuestro `docs/PERFORMANCE.md`: ~0,45 ms por `hMerkle`, ~110 ms por inserción en un SMT de profundidad 254, ~440 ms por `submit` (≈ 2 tx/s por núcleo). La suite `test:protocol` tarda ~170 s en mi entorno.

**Mejoras, de mayor a menor impacto:**

1. **Hojas con atajo (estilo Jellyfish/extension nodes).** Hoy cada hoja nueva cuesta ~254 hashes. Guardando la hoja en la profundidad de divergencia, el coste pasa a ~log2(N) (≈ 20 con un millón de cuentas, estimación). Cambia todas las raíces, por lo que conviene hacerlo ahora, antes de 1.0, con migración de snapshot (ya tenéis el mecanismo).
2. **Actualización de raíz por lotes.** Calcular una sola vez los nodos superiores compartidos por todas las hojas de un bloque.

3. **Poseidon nativo/WASM correcto** (ver I2) con Montgomery de 4×64 bits; vuestra medición de ~8 % en JS indica que el cuello es el uso de `bigint`.
4. **Separar validar de aplicar** y paralelizar la verificación Ed25519 en un pool de workers.
5. **Tests:** compartir un ledger base por snapshot en lugar de reconstruirlo, y ejecutar en paralelo los archivos independientes.

## 6. Funcionalidades recomendadas

- **Ejecución determinista de extremo a extremo:** sin aleatoriedad ni lectura de entorno en transiciones (I1), con test diferencial TS↔Rust.
- **Cuentas:** rotación de claves, multi-cuenta por mnemónico, umbral M-de-N y recuperación (I4).
- **Fuzzing y propiedades** sin dependencias: generadores con PRNG con semilla sobre `node:test` para `submit`, snapshots y migraciones; invariantes de conservación de valor y de oferta.
- **Modelo formal** del consenso de los labs (TLA+/Apalache) antes de promoverlos fuera de `src/lab`.
- **Observabilidad:** logs estructurados sin secretos, métricas (cola, altura, latencia de submit) y alertas del productor de altura.
- **API:** OpenAPI, errores con códigos estables, idempotencia en todas las escrituras.
- **Snapshots:** firma con umbral en lugar de una clave única.

## 7. Plan priorizado

### P0 (esta semana)

- Aplicar `fix-supplycap.patch` y fusionar el test de regresión (C1).
- Dejar el CI verde: mover `process.env` fuera de `scale-sizes.ts` y decidir la solución de I1 (C2).
- Límites de cuerpo por ruta y validación numérica en la API; respuesta 500 genérica (C3, C4).

### P1 (este mes)

- Rediseño de mint con compromiso aportado por el cliente (I1).
- Corregir o retirar el generador WASM, con test de paridad (I2).
- Endurecer el vault (I3) y la superficie HTTP (I8).
- CI: SHA pinning, `tsc` bloqueante, CodeQL, gitleaks, `cargo deny` (I9).
- Actualizar documentación y generarla desde fuentes (I10).

**P2 (antes de 1.0)**

- Hojas con atajo en el SMT y actualización por lotes (sección 5).
- Derivación SLIP-0010, rotación de claves y umbral de cuentas (I4).
- Decidir el esquema de prueba y la política de ceremonia (I12).
- Auditoría externa del ledger completo, marketplace, settlement y circuitos.

# 8. Anexo: reproducción

```
npm run lint:determinism      # sale con código 1, 3 violaciones
npm run test:protocol         # 210 tests: 208 ok, 2 fallan (lint)
node --experimental-strip-types --no-warnings --test src/testnet/ledger-mint-
reservation-cap.test.ts
patch -p0 < fix-supplycap.patch   # ajustad -p según la ruta de vuestro árbol
```

Archivos tocados por el parche: `src/testnet/ledger.ts` (dos cambios) y `src/testnet/ledger-mint-reservation-cap.test.ts` (nuevo).